

Fanchin Alberto 244395

Titolo del progetto: TeamSync

Tecnologie utilizzate: Kotlin · Jetpack Compose · Android SDK 24+ · Ktor Client · Node.js · Express · SQLite (better-sqlite3)

1) Introduzione

Le squadre sportive amatoriali — pallavolo, basket, calcio a livello dilettantistico — operano con risorse organizzative estremamente limitate. La gestione quotidiana del gruppo (convocazioni, conferme di presenza, organizzazione dei trasporti verso le trasferte, definizione delle rotazioni in campo, monitoraggio dei livelli tecnici individuali) è tipicamente affidata a strumenti generalisti e frammentati: gruppi WhatsApp per le comunicazioni, fogli Excel condivisi per i ruoli, scambi vocali per gli spostamenti, fogli di carta per le formazioni.

Il NEED identificato è dunque l'assenza di uno strumento *verticale*, integrato e mobile-first, pensato per il flusso operativo reale di una squadra amatoriale, in cui un allenatore-volontario e un capitano gestiscono, con tempo limitato e senza budget, gli atleti.

L'obiettivo di TeamSync è fornire un'applicazione Android nativa che concentri in un'unica esperienza le quattro attività ad alta frequenza di una stagione amatoriale:

1. Convocazione e RSVP alla singola partita, tramite un'interazione gestuale rapida (swipe-card) che permette di rispondere in meno di due secondi dalla notifica.
2. Allenamento auto-somministrato sulla base dei gap tecnici dichiarati dall'atleta (carta "Daily Drill" generata server-side a partire dal campo skillgaps del profilo utente).
3. Carpooling per le trasferte, con una pianificazione di percorso (variante semplificata del Vehicle Routing Problem) calcolata lato backend tramite la soluzione euristica del nearest-neighbor.
4. Lavagna tattica interattiva per la gestione delle rotazioni di pallavolo (altri sport purtroppo molto complessi da realizzare), con drag-and-drop libero dei giocatori sul campo.

L'applicazione è pensata per essere offerta gratuitamente, eseguita interamente su infrastruttura locale durante la fase prototipale (backend Node.js esposto su LAN)

2) Identificazione del segmento utente

Il segmento utente target di TeamSync è costituito da squadre sportive amatoriali di pallavolo, basket e calcio a 5/11, con priorità per la pallavolo (sport che ha guidato la modellazione delle funzionalità tattiche e del quale possiedo conoscenze più approfondite). All'interno di questo campione, l'utente principale è doppio:

- Il giocatore amatoriale (18–45 anni), che pratica lo sport 1–3 volte a settimana in modo continuativo ma non professionale, possiede uno smartphone di fascia media (Android 7+)

e dispone di tempo limitato per la gestione amministrativa della squadra. Cerca uno strumento intuitivo, facile e che minimizzi il numero di tap per ogni operazione ricorrente (conferma presenza, richiesta di passaggio in auto, consultazione della rotazione).

- Il capitano / allenatore-volontario, che oltre al ruolo di sportivo si sobbarca le mansioni gestionali: deve sapere rapidamente quanti atleti hanno confermato, organizzare il trasporto in modo equo, comunicare la formazione iniziale, e tenere traccia delle aree tecniche da allenare. Cerca uno strumento che fornisca tool che lo aiutino, come la lasagna o la visualizzazione presenze.

La scelta di concentrarsi su questo profilo è motivata da una lacuna concreta: il mercato delle app sportive è polarizzato su due estremi: la *fitness app individuale* (Strava, MyFitnessPal) e la *piattaforma gestionale professionale* (Hudl, TeamSnap a pagamento). Tra i due estremi non esiste un'esperienza pensata per il *micro-team* italiano dilettantistico, dove il rapporto fra atleti e tempo organizzativo è sbilanciato e dove l'impegno necessario all'utilizzo sia basso. Le esigenze specifiche di questo profilo che TeamSync indirizza sono:

- **Decisione in mobilità in pochi secondi:** l'utente legge la notifica della partita mentre si sposta. La swipe-card sul DashboardScreen permette di rispondere senza aprire menu intermedi.
- **Auto-organizzazione del trasporto:** il backend espone un endpoint POST `/api/carpools/:matchId/optimize` che ordina i pickup secondo nearest-neighbor, eliminando l'organizzazione manuale nei gruppi di messaggistica.
- **Autovalutazione tecnica continua:** lo schermo ProfileScreen con sei slider Compose (Spiking, Setting, Digging, Serving, Blocking, Passing) salva il livello percepito sul backend, che lo riusa nella selezione del drill quotidiano.
- **Tatticismo accessibile:** il TacticsScreen è disegnato per essere comprensibile anche da un capitano senza formazione da allenatore professionista, con visualizzazione del campo a colori per ruolo (S, OH, MB, OPP, L) e rotazione automatica P1→P6.

3) Stato dell'arte

Il panorama Play Store presenta diverse categorie di applicazioni potenzialmente comparabili, nessuna delle quali coincide con il **NEED** di TeamSync:

- **TeamSnap, SportEasy, Spond:** piattaforme gestionali multi-sport con maturità elevata (milioni di download, presenza pluriennale). Sono però *generaliste*, orientate a un modello SaaS a pagamento (TeamSnap) o freemium con limitazioni stringenti (Spond).

Ma soprattutto, non offrono funzionalità tattiche specifiche come la lavagna delle rotazioni di pallavolo né suggerimenti tecnici personalizzati. La loro UX è inoltre pensata per il modello USA dello youth sport (con ruoli "parent", "team manager") che non rappresenta il contesto italiano amatoriale adulto, dove spesso l'allenatore è anch'esso un giocatore.

- **Carpool / RideShare app (BlaBlaCar, scopo generale):** risolvono il problema del trasporto ma non sono integrate con il contesto sportivo, l'utente dovrebbe coordinare manualmente due app distinte.

- **Volleyball Tactic Board, Coach's Eye e applicazioni simili:** si concentrano sulla sola lavagna tattica, spesso come strumento da allenatore professionista, senza la dimensione gestionale della rosa, come già detto, l'obiettivo di TeamSync è fornire un sistema che sia "tutto incluso".
- **Strumenti generici (Google Calendar + WhatsApp + Google Sheets):** rappresentano l'attuale strumento delle squadre amatoriali, con maturità massima ma costo cognitivo elevato per l'utente.

Tra le applicazioni con un livello di maturazione *elevato* (Spond e TeamSnap, in particolare) il limite principale è la mancanza di integrazione verticale: il modulo trasporti non comunica con quello presenze e la dimensione tattica è assente. Sul lato opposto, le applicazioni tattiche o di carpooling singolo soffrono di una *frammentazione funzionale* che riproduce la frammentazione attuale degli strumenti generici.

Il posizionamento di TeamSync è quindi quello di un'applicazione integrata, verticale sul micro-team amatoriale, gratuita e self-hosted nella fase prototipale, che combina in un'unica esperienza coerente le quattro funzionalità chiave (RSVP, drill personalizzato, carpooling con VRP, tattica). Nessuna delle alternative esaminate copre simultaneamente queste quattro aree con un'esperienza utente unificata.

4) Architettura

TeamSync segue un'architettura client-server a due livelli con comunicazione HTTP/JSON e separazione netta delle responsabilità tra presentazione (Android nativo), logica applicativa (Node.js) e persistenza (SQLite).

Diagramma generale dell'architettura:

Inserito come immagine nel repository github

Descrizione dei componenti:

-Client Android : Layer di presentazione.

L'intera UI è costruita con Jetpack Compose e Material 3.

Il tema applicativo (Theme.kt) definisce una `darkColorScheme` con colore primario Lime = `Color(0xFF39FF14)` e superficie Bg950(Grigio) = `Color(0xFF0A0A0A)`, applicata globalmente attraverso `MaterialTheme`.

La navigazione è gestita da Navigation Compose con un `NavHost` centrale in `MainActivity`, che instrada cinque destinazioni autenticate (Dashboard, Carpool, Tactics, Profile, EventDetail con argomento `matchId: Int`) e tre pre-autenticazione (Welcome, Login, Register).

Interazioni gestuali. Due schermate fanno uso intensivo del sistema di `pointer event` di Compose:

- **DashboardScreen** integra una *swipe-card* implementata con `Modifier.pointerInput { detectHorizontalDragGestures(...) }`. Lo spostamento orizzontale è animato tramite `Animatable<Float>`, traduce il delta in `translationX` e `rotationZ` via `graphicsLayer`, e al superamento di una soglia (110 dp) emette uno stato "in" o "out" che viene registrato sul

backend tramite `POST /api/matches/:id/rsvp`. Il colore di fondo della card è in funzione dello stato `match.my_rsvp` (grigio se null, gradiente verde se "in", gradiente rosso se "out"), garantendo che la scelta sia visibile, chiara e persistente dopo lo swipe.

- **TacticsScreen** realizza la lavagna tattica con un Canvas di Compose per le linee del campo (perimetro, linee di attacco a 3 m, rete) e con sei Box posizionati via `Modifier.offset` per i giocatori.

Il drag libero è implementato con `Modifier.pointerInput(player.pos) { detectDragGestures { change, dragAmount -> ... } }`.

La matematica del drag è incrementale sullo stato corrente ($rx + dx/widthPx$ invece che ricalcolata da uno snapshot), così le posizioni inseguono fluidamente il dito anche durante gesti prolungati.

-Layer ViewModel.

Ogni schermata significativa è supportata da un ViewModel distinto che espone uno `StateFlow<UiState>` immutabile e raccoglie le interazioni utente come funzioni pubbliche.

Le coroutine vengono lanciate su `viewModelScope`, garantendo la cancellazione automatica al di fuori del ciclo di vita. Il pattern adottato è classico MVVM unidirezionale: l'UI osserva lo stato via `collectAsState()`, invoca funzioni del VM, e il VM emette nuovi stati immutabili. Non viene introdotta una libreria di dependency injection (Hilt / Koin) per mantenere il footprint del prototipo contenuto: la dipendenza tra VM e `ApiClient` è risolta in `ServiceLocator`.

-Layer dati lato client.

L' `ApiClient` è basato su `Ktor Client` con plugin `ContentNegotiation` e serializzazione `kotlinx.serialization`.

Il `defaultRequest` aggiunge automaticamente `Content-Type: application/json`, `accept` e l'header `Authorization: Bearer <jwt>` letto da `SessionStore`. Quest'ultimo è un wrapper su `DataStore Preferences` che mantiene il token e il profilo utente minimale, esposto come `StateFlow` per consentire alle schermate di reagire a logout o cambio sessione.

L'URL base è iniettato a compile time via `BuildConfig.API_BASE_URL = "http://10.0.2.2:4000"` e il `network_security_config.xml` autorizza cleartext esclusivamente per 10.0.2.2, localhost e 127.0.0.1, in modo da non aprire vulnerabilità sui dispositivi di produzione.

-Backend Node.js.

L'API server è un singolo processo Express 5 (`server.js`) che monta `cors()`, `express.json()` e un middleware `auth()` HS256 implementato manualmente con `crypto.createHmac('sha256', SECRET)`. Il backend non dipende da framework ORM: usa `better-sqlite3` in modalità sincrona, sfruttando la WAL mode per concorrenza in lettura. All'avvio il server esegue `schema.sql` con `CREATE TABLE IF NOT EXISTS` e applica migrazioni forward-only (`ALTER TABLE ... ADD COLUMN`) avvolte in try/catch per non incorrere in errori dati database preesistenti.

-Note sugli algoritmi utilizzati:

- L'endpoint `POST /api/carpools/:matchId/optimize` implementa una variante didattica del Vehicle Routing Problem: per ogni carpool carica i passeggeri con coordinate pickup_lat/Ing, parte dall'origine dell'autista, e seleziona ricorsivamente il passeggero più vicino (distanza euclidea in gradi) finché tutti i pickup non sono visitati, terminando al luogo della partita. Il risultato è salvato come JSON nel campo route_json e gli stop_order dei passeggeri vengono aggiornati in una db.transaction. L'euristica è volutamente semplice ($O(n^2)$ per veicolo).
- L'endpoint `GET /api/drills/today` materializza un join tra users.skill_gaps (array JSON delle lacune dichiarate) e drills.skill_tag: la query estrae casualmente un drill che corrisponde a uno dei gap dell'utente, con fallback su un drill di default se nessuna corrispondenza è disponibile.

-Persistenza.

Lo schema SQLite normalizza nove entità principali:

- users (con campi skill_gaps e skill_levels come JSON)
- teams
- team members
- matches (con venue, address, coordinate, stato e punteggi)
- rsvps (vincolo UNIQUE(match_id, user_id))
- carpools
- carpool passengers
- drills
- rotations (con positions come JSON serializzato).

Gli indici secondari (idx_matches_team_kickoff, idx_rsvps_match, idx_drills_sport_skill) ottimizzano i pattern di query più frequenti del client.

-Servizi esterni.

L'attuale architettura non si appoggia a servizi cloud: tutto gira in locale durante la fase di sviluppo. I punti di estensione futuri sono identificati:

1. sostituzione dell'euristica VRP con Google OR-Tools o Mapbox Optimization API
2. integrazione con Firebase Cloud Messaging per le notifiche push di nuove convocazioni
3. sostituzione del backend Node.js con Cloud Run + Cloud SQL per il deploy produttivo.

Il contratto JSON e l'ApiClient Ktor sono progettati per essere indipendenti dall'implementazione server.

5) Analisi critica dei possibili limiti dell'applicazione

L'analisi dei limiti del prototipo è stata suddivisa in macro aree, così da permetterne una dettagliata analisi.

1. **Sicurezza e autenticazione:** L'implementazione attuale del JWT è di tipo didattico, utilizza un secret HS256 hard-coded nel codice (`SECRET = process.env.JWT_SECRET || 'dev-secret-change-me'`) e non implementa rotazione del token né refresh_token.

Il traffico è in cleartext HTTP per supportare l'emulatore Android su 10.0.2.2, con la `network_security_config.xml` esplicitamente configurata su tre host fidati.

In produzione sarebbe necessario:

- a) TLS terminato da un reverse proxy (Caddy / Nginx)
- b) secret iniettati via variabili d'ambiente o servizio di gestione segreti
- c) introduzione di `refresh_token` con scadenza differenziata
- d) rate limiting su `/api/auth/login` per resistere a credential stuffing.

2. **Ottimizzazione del Vehicle Routing Problem:** L'euristica nearest-neighbor produce risultati subottimali nei casi con più di 5–6 passeggeri. Inoltre la distanza euclidea in gradi geografici approssima la realtà solo a scala locale (qualche chilometro) e ignora la rete stradale reale, i sensi unici, i tempi di percorrenza.

Estensioni possibili: integrare l'API Directions di OSRM/Mapbox per il calcolo della matrice di costo e adottare un solver Clarke-Wright savings o Or-opt per il refining.

Inoltre il modello attuale non considera vincoli temporali, ovvero il fatto che un passeggero possa essere preso solo a partire da un certo orario, né preferenze (atleta che predilige un autista specifico).

3. **Funzionalità tattiche limitate:** La lavagna delle rotazioni è funzionale ma essenziale: supporta la rotazione oraria della pallavolo, non permette la visualizzazione dei percorsi di traiettoria del pallone e non permette al momento di salvare scenari multipli.

Estensioni possibili: è un editor di scenari tattici riutilizzabili (rotazioni tipizzate per fase: ricezione del servizio, attacco dopo difesa, copertura), con possibilità di registrare animazioni step-by-step. Inoltre non sono coperti altri sport.

4. **Esperienza offline e sincronizzazione:** L'applicazione presuppone connettività verso il backend per ogni interazione significativa: non c'è un layer di cache locale dei dati né una coda di richieste differite. Se l'utente prova a fare RSVP in una qualsiasi situazione senza segnale, l'operazione fallisce silenziosamente.

Estensioni possibili: introduzione di Room come *single source of truth* lato client, con un WorkManager che drena le mutazioni accumulate verso il backend al ripristino della connessione, e una strategia di conflict resolution (last-write-wins basato sul timestamp `responded_at`).

5. **Scalabilità del backend e modello dati:** L'uso di SQLite con better-sqlite3 in modalità sincrona è ottimale per il deploy single-tenant (una squadra, un'istanza), ma non scala oltre un manipolo di team perché ogni query blocca il singolo thread Node.

Estensioni possibili: per supportare più team in multi-tenancy sarebbero necessari:

- a) migrazione a PostgreSQL con pool di connessioni asincrono
- b) introduzione di un `tenant_id` come chiave di partizionamento nelle entità principali
- c) sostituzione dello stato in-memory dei ViewModel con caching server-side (Redis) per le query ad alta frequenza come `GET /api/matches/upcoming`.

Inoltre il modello `rotations.positions` come blob JSON limita la capacità di interrogare formazioni per ruolo o per atleta in `cross-team analytics`.

Estensioni possibili: estrarre le posizioni in una tabella normalizzata `rotation_slots`.

6. **Accessibilità e internazionalizzazione:** Il prototipo non implementa stringhe localizzate (sono hard-coded nelle `Composable`), né dichiara `contentDescription` significativi sulle icone decorative.

Estensioni possibili: per la pubblicazione su Play Store sarebbero necessari:

- a) file `strings.xml` con varianti `values-it/`, `values-en/`
- b) controllo via TalkBack di tutti i flussi gestuali (con particolare attenzione alla `swipe-card`, che ha bisogno di un'azione semantica equivalente per chi naviga con screen reader)
- c) verifica del contrasto del verde lime su sfondo nero rispetto alle linee guida WCAG AA per testo piccolo.

6) Conclusioni

Riassumendo, TeamSync nel suo stato attuale dimostra la fattibilità tecnica dell'idea, ma rimane pur sempre un prototipo.

Le direzioni di estensione individuate sono i possibili step successivi, resi possibili dal disegno architetturale a layer disaccoppiati (`Compose` → `ViewModel` → `ApiClient` → `Express` → `SQLite`), esplicitamente pensato per assorbire queste evoluzioni senza riscritture sostanziali del codice esistente.